SaTScan in Practice — COVID-19 Deaths in Pennsylvania

BSTA 374 · Cluster detection · class demonstration

Eric M. Delmelle · Lehigh University

2026-10-15

> **Today**
>
> By the end of the class you should be able to
>
> 1. explain in plain words how the spatial scan statistic finds a cluster;
> 2. prepare the three input files SaTScan needs and run a purely spatial Poisson scan;
> 3. read the output: observed and expected deaths, relative risk, LLR and p-value;
> 4. say what a significant cluster does, and does not, tell us.
>
> The SaTScan background page has more detail and a second worked example.

Packages for today:

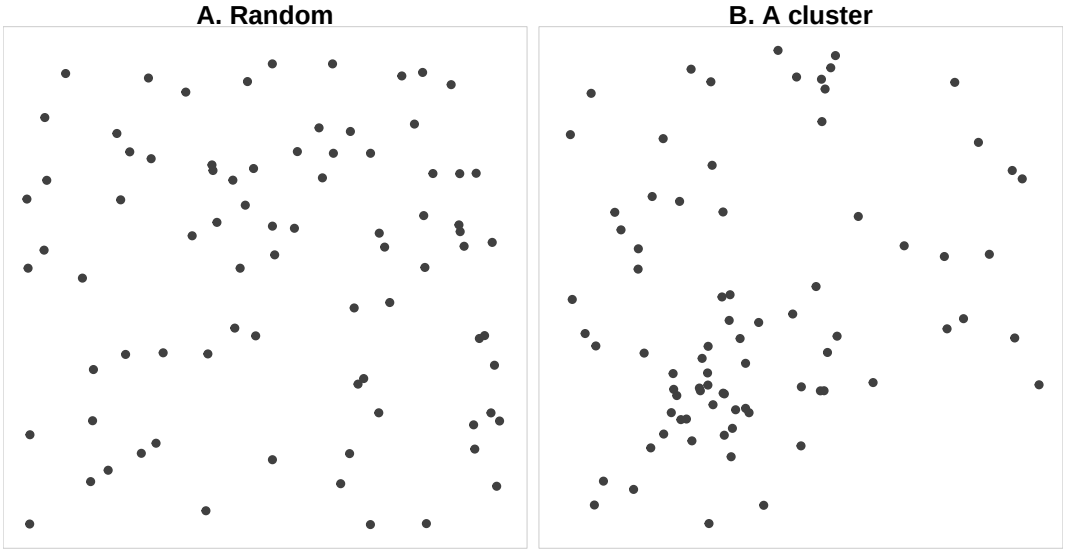
```
library(sf)
library(dplyr)
library(readr)
library(ggplot2)
library(patchwork)
library(tigris)                    # county boundaries
library(SpatialEpi)                # kulldorff()
options(tigris_use_cache = TRUE)
```

The question

Cases and deaths are never spread perfectly evenly. Map A below places points at random; map B adds a small group of extra points in the lower left. Our eyes find "clusters" in both. The question for today is whether a concentration is **larger than chance alone would produce**. The spatial scan statistic answers it with a formal test.

```r
set.seed(42)
pts <- bind_rows(
  tibble(map = "A. Random",    x = runif(80), y = runif(80)),
  tibble(map = "B. A cluster", x = c(rnorm(25, 0.3, 0.06), runif(55)),
                               y = c(rnorm(25, 0.3, 0.06), runif(55))))

ggplot(pts, aes(x, y)) +
  geom_point(colour = "grey25", size = 1.3) +
  facet_wrap(~ map) +
  coord_equal() +
  theme_void() +
  theme(strip.text   = element_text(face = "bold", size = 12),
        panel.border = element_rect(fill = NA, colour = "grey80"))
```



From LISA to the scan statistic

Local Moran's I asked a question about **each county**: *is this county's SMR similar to its neighbours'?* The spatial scan statistic asks a question about **groups of counties**: *is there a group of neighbouring counties where deaths are higher than expected?*

The method is implemented in **SaTScan**, free software written by Martin Kulldorff (satscan.org) and used since 1997 by the CDC, state health departments and researchers.

What SaTScan can scan for

Type of scan	Question	In this course
Purely spatial	*Where* are there more cases than expected? Time is ignored.	**today**
Purely temporal	*When* were there more cases? Space is ignored.	
Space–time	Where *and* when? Typical of outbreaks.	later
Prospective space–time	Is an outbreak emerging *now*?	surveillance weeks

Probability models

Model	Data needed	Typical use
Poisson	case counts and population (or expected counts)	**today**: deaths by county
Bernoulli	cases and controls (0/1)	case–control studies
Space–time permutation	case counts only, no population	rapid outbreak detection
Normal	a continuous outcome	e.g. birth weight, exposure levels

Today we run a **purely spatial** scan with the **Poisson** model, looking for **high rates** only.

The data

We use the 2020 COVID-19 deaths in Pennsylvania's 67 counties, with the same steps as before: join, project, expected counts.

```
covid <- read_csv("data/pa_covid_deaths_2020.csv", show_col_types = FALSE) |>
  mutate(GEOID = sprintf("%05d", as.integer(FIPS)))
pa <- counties(state = "PA", cb = TRUE, year = 2020, progress_bar = FALSE)

pa_covid <- pa |>
  left_join(covid, by = "GEOID") |>
  st_transform(5070) |>
  arrange(GEOID)

r <- sum(pa_covid$deaths_2020) / sum(pa_covid$Population)   # state rate
pa_covid <- pa_covid |>
  mutate(E   = r * Population,                              # expected deaths
```

```
         SMR = deaths_2020 / E)

nrow(pa_covid)                      # 67 counties
```

```
[1] 67
```

```
sum(pa_covid$deaths_2020)           # deaths in 2020
```

```
[1] 16030
```

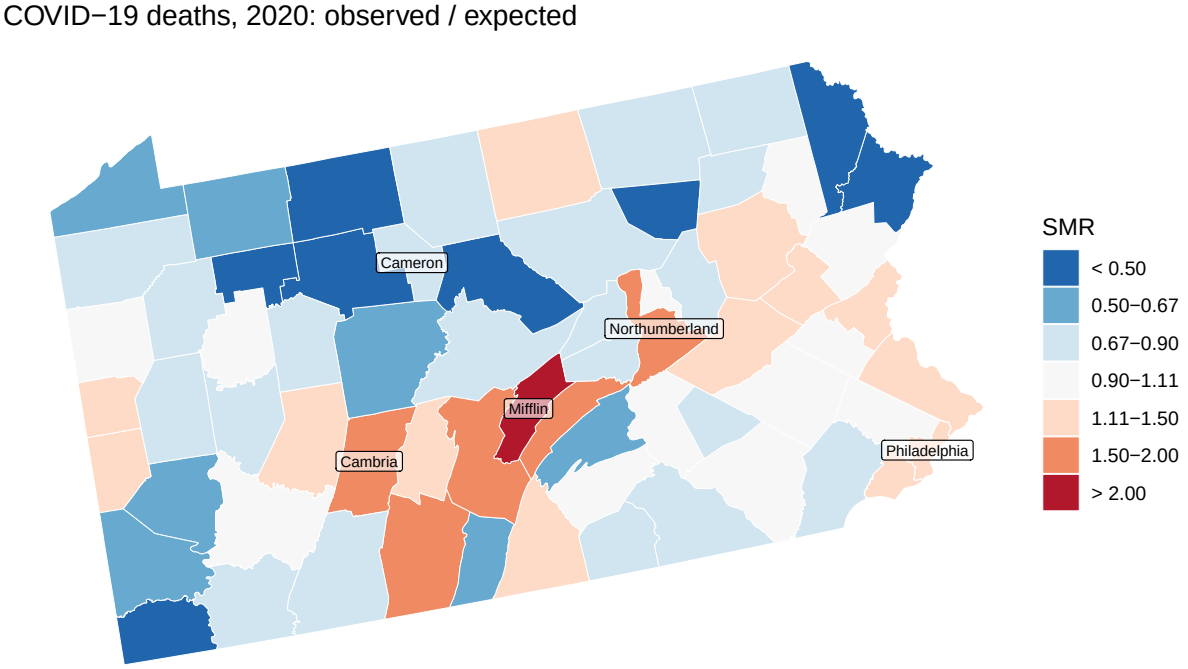
The pattern we start from

The SMR map, with the same classes and colours as your Exercise 4 map: red counties had more deaths than expected, blue fewer, and the palest class is "about as expected".

```
pa_covid <- pa_covid |>
  mutate(smr_class = cut(SMR,
                         breaks = c(0, 0.5, 2/3, 0.9, 1/0.9, 1.5, 2, Inf),   # centred on 1
                         labels = c("< 0.50", "0.50-0.67", "0.67-0.90", "0.90-1.11",
                                    "1.11-1.50", "1.50-2.00", "> 2.00")))

# a few counties we will talk about
named <- filter(pa_covid, NAME %in% c("Philadelphia", "Northumberland", "Mifflin",
                                      "Cambria", "Cameron"))

ggplot(pa_covid) +
  geom_sf(aes(fill = smr_class), colour = "white", linewidth = 0.15) +
  geom_sf_label(data = named, aes(label = NAME), size = 2.4,
                fill = alpha("white", 0.6)) +                          # see-through labels
  scale_fill_brewer(palette = "RdBu", direction = -1, drop = FALSE,
                    name = "SMR") +                                       # red = above 1
  labs(title = "COVID-19 deaths, 2020: observed / expected") +
  theme_void()
```



The highest SMRs are in central Pennsylvania, around Mifflin, its neighbour Juniata and Northumberland, and further west around Cambria. Philadelphia and several of its neighbours are only pale red. **Before we run the scan: where do you expect it to find clusters?**

How the scan works: three ideas

Idea 1 — A circle that grows

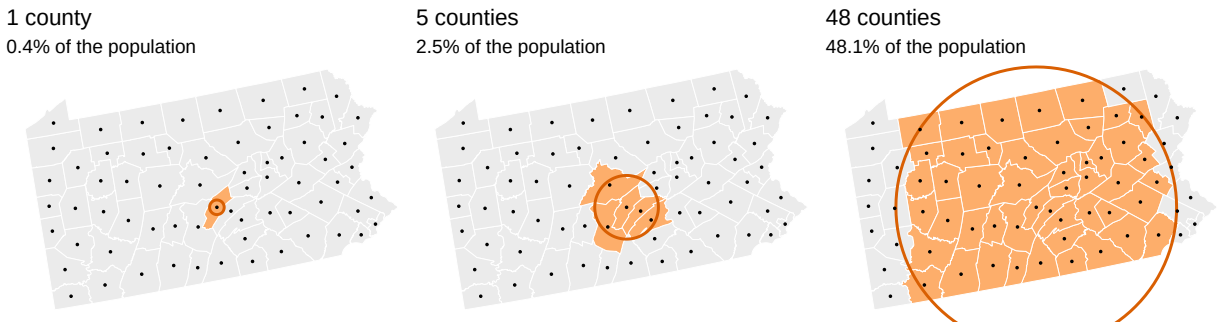
Place a circle on a county centroid. Start with a radius that holds only that county, then let it grow, taking in the next-nearest county each time, until the circle holds a set share of the state's population: **50%** by default. Repeat at every county centroid. A county is *inside* a circle when its centroid is. Each circle is a **candidate cluster**.

```
cen   <- st_centroid(st_geometry(pa_covid))
ctr   <- cen[pa_covid$NAME == "Mifflin"]                    # circle centre
d_km  <- as.numeric(st_distance(cen, ctr)) / 1000           # km to centre
ord   <- order(d_km)
share <- cumsum(pa_covid$Population[ord]) / sum(pa_covid$Population)  # population inside
k_max <- max(which(share <= 0.5))                           # largest circle allowed
bb    <- st_bbox(pa_covid)

window_map <- function(k) {
  r_k <- mean(d_km[ord][k:(k + 1)])                         # radius reaching k counties
  ggplot() +
```

```
    geom_sf(data = pa_covid, aes(fill = d_km <= r_k), colour = "white", linewidth = 0.15) +
    geom_sf(data = st_buffer(ctr, r_k * 1000), fill = NA, colour = "#D95F02", linewidth = 0.8) +
    geom_sf(data = cen, size = 0.3) +
    scale_fill_manual(values = c(`FALSE` = "grey92", `TRUE` = "#FDAE6B"), guide = "none") +
    coord_sf(xlim = bb[c(1, 3)], ylim = bb[c(2, 4)]) +
    labs(title = paste(k, if (k == 1) "county" else "counties"),
         subtitle = sprintf("%.1f%% of the population", 100 * share[k])) +
    theme_void()
}

window_map(1) + window_map(5) + window_map(k_max)
```



Three of the circles centred on Mifflin. The last one is the largest allowed: it takes in 48 of the 67 counties.

Idea 2 — Inside versus outside: the likelihood ratio

For each circle, compare the deaths **inside** with the deaths **outside**. With the Poisson model the comparison is the log-likelihood ratio (LLR):

$$\text{LLR} = c \log \frac{c}{E} + (C - c) \log \frac{C - c}{C - E} \qquad \text{if } c > E, \text{ otherwise } 0,$$

where c is the number of deaths observed inside the circle, E the number expected inside it at the state rate, and C the total number of deaths in Pennsylvania. In plain words, the LLR measures how much better the data are explained by *a higher rate inside the circle* than by *one rate everywhere*. A circle with no excess gets 0.

We can compute it by hand for five single counties. The code follows the formula letter by letter: `c` for observed deaths, `E` for expected deaths, `C` for the state total.

```
C <- sum(pa_covid$deaths_2020)                    # total deaths in PA

five <- pa_covid |>
  st_drop_geometry() |>
  filter(NAME %in% c("Philadelphia", "Northumberland", "Mifflin", "Juniata", "Cameron")) |>
```

```r
  transmute(county = NAME,
            c      = deaths_2020,                  # observed deaths
            E      = E,                            # expected deaths
            excess = c - E,                        # extra deaths
            SMR    = c / E,
            LLR    = if_else(c > E,                # the formula
                             c * log(c / E) + (C - c) * log((C - c) / (C - E)),
                             0)) |>
  arrange(desc(LLR))

knitr::kable(five, digits = 2)                     # print as a table
```

county	c	E	excess	SMR	LLR
Philadelphia	2450	1983.48	466.52	1.24	58.84
Northumberland	226	113.75	112.25	1.99	43.30
Mifflin	119	57.77	61.23	2.06	24.88
Juniata	48	31.01	16.99	1.55	3.99
Cameron	4	5.57	-1.57	0.72	0.00

Mifflin has the highest SMR in the state, yet Philadelphia, with a much smaller SMR, has the larger LLR. Philadelphia's excess of about 467 deaths is much harder to explain by chance than Mifflin's excess of about 61. The scan works with **counts**, not ratios: a modest excess in a large population can be stronger evidence than a high SMR in a small county. Cameron, with fewer deaths than expected, gets an LLR of 0.

The scan computes the LLR for every circle. The circle with the **largest LLR** is the **most likely cluster**.

Idea 3 — Could the largest LLR be chance? Monte Carlo

Searching thousands of circles, we will always find *some* circle with a large LLR. The question is whether ours is larger than what a map with no cluster would give. SaTScan answers it by simulation:

1. Redistribute the 16,030 deaths at random among the counties, in proportion to their expected deaths. This is a map with **no cluster**.
2. Run the whole scan on that map, every circle, and keep its largest LLR.
3. Repeat 999 times.
4. $p = \dfrac{\text{number of random maps whose largest LLR} \geq \text{ours} \; + 1}{999 + 1}$

Because each random map is also searched over every circle, the test already accounts for having looked at thousands of circles. Unlike LISA, there is no need for a separate multiple-testing adjustment.

Part 1 — SaTScan software

Step 1. Write the three input files

SaTScan reads plain text files linked by a **location ID**. We use the county `GEOID`.

File	Columns	Here
Case file `.cas`	location ID, number of cases	GEOID, deaths in 2020
Population file `.pop`	location ID, year, population	GEOID, 2020, population
Coordinates file `.geo`	location ID, latitude, longitude	GEOID, county centroid

```
dir.create("satscan", showWarnings = FALSE)    # folder for the files

# county centroids in latitude / longitude
ll <- pa_covid |>
  st_centroid() |>
  st_transform(4326) |>                         # SaTScan wants lat/long
  st_coordinates()

# one plain table with everything SaTScan needs
ss <- pa_covid |>
  st_drop_geometry() |>
  mutate(year = 2020,
         lat  = round(ll[, "Y"], 5),
         lon  = round(ll[, "X"], 5))

# three files, separated by spaces, no header
write_delim(select(ss, GEOID, deaths_2020),      "satscan/pa_covid.cas", col_names = FALSE)
write_delim(select(ss, GEOID, year, Population), "satscan/pa_covid.pop", col_names = FALSE)
write_delim(select(ss, GEOID, lat, lon),         "satscan/pa_covid.geo", col_names = FALSE)

# first three lines of each file
readLines("satscan/pa_covid.cas", n = 3)
```

```
[1] "42001 88"   "42003 1009" "42005 69"
```

```
readLines("satscan/pa_covid.pop", n = 3)
```

```
[1] "42001 2020 103009"  "42003 2020 1216045" "42005 2020 64735"
```

```
readLines("satscan/pa_covid.geo", n = 3)
```

```
[1] "42001 39.87147 -77.2179"  "42003 40.46868 -79.98121"
[3] "42005 40.81193 -79.46461"
```

⚠ Two common mistakes

- The coordinates file lists **latitude, then longitude**. Swapping them puts Pennsylvania in Antarctica, and SaTScan will still run.
- The location IDs must be **identical** in all three files. A county whose ID does not match is dropped, without an error message.

Step 2. Set up the analysis in the GUI

Open SaTScan and choose **Create New Session**.

Input tab

- *Case File*: `pa_covid.cas`. In the import wizard, map column 1 to *Location ID* and column 2 to *Number of Cases*.
- *Time Precision*: **None**. *Study Period*: 2020/01/01 to 2020/12/31.
- *Population File*: `pa_covid.pop` (Location ID, Date/Time, Population).
- *Coordinates File*: `pa_covid.geo`, *Coordinates*: **Lat/Long**.

Analysis tab

- Type of analysis: **Purely Spatial**.
- Probability model: **Poisson**.
- Scan for areas with: **High Rates**.
- *Advanced, Spatial Window*: maximum cluster size **50 percent of the population at risk**; window shape **circular** (elliptic is the alternative for elongated clusters).
- *Advanced, Inference*: **999** Monte Carlo replications.

Output tab

- Main results file: `satscan/pa_covid_results.txt`.
- Optional: tick *Google Earth (KML)* to view the circles, and the shapefile option to map the clusters in R or ArcGIS.

Click **Run**.

Step 3. Read the results file

SaTScan lists the **most likely cluster** first, then the **secondary clusters**. By default it reports only secondary clusters that do **not overlap** an earlier one. For each cluster:

In the SaTScan results file	Meaning	In R, `kulldorff()`
Location IDs included	the counties inside the circle	`location.IDs.included` (row numbers)
Coordinates / radius	centre and size of the circle	—
Population	people inside the circle	`population`
Number of cases	deaths observed inside, c	`number.of.cases`

In the SaTScan results file	Meaning	In R, `kulldorff()`
Expected cases	deaths expected inside at the state rate, E	`expected.cases`
Observed / expected	c/E: inside compared with the state rate	`SMR`
Relative risk	rate inside divided by rate outside	computed below (`RR`)
Log likelihood ratio	strength of the evidence (Idea 2)	`log.likelihood.ratio`
P-value	from the 999 random maps (Idea 3)	`p.value`

Part 2 — The same scan in R

`SpatialEpi::kulldorff()` runs the same Poisson scan without leaving R. It needs coordinates on a flat grid, so we use the projected centroids in kilometres.

```
cen_km <- st_coordinates(st_centroid(pa_covid)) / 1000   # Albers, km

set.seed(374)
kd50 <- kulldorff(geo             = cen_km,
                  cases           = pa_covid$deaths_2020,
                  population      = pa_covid$Population,
                  expected.cases  = pa_covid$E,          # Poisson model
                  pop.upper.bound = 0.5,                 # max 50% of population
                  n.simulations   = 999,
                  alpha.level     = 0.05,
                  plot            = FALSE)
```

`kulldorff()` evaluated 2,570 circles (Idea 1). Its result is a list: `kd50$most.likely.cluster` holds the first cluster and `kd50$secondary.clusters` the others, each with the fields listed in Step 3. For example:

```
kd50$most.likely.cluster$number.of.cases       # observed deaths
```

```
[1] 8250
```

```
kd50$most.likely.cluster$expected.cases        # expected deaths
```

```
[1] 7092.768
```

```
kd50$most.likely.cluster$p.value                # p-value
```

```
[1] 0.001
```

Two helper functions collect these fields into a table and a map. You do not need to understand them to use them; expand the code to copy it.

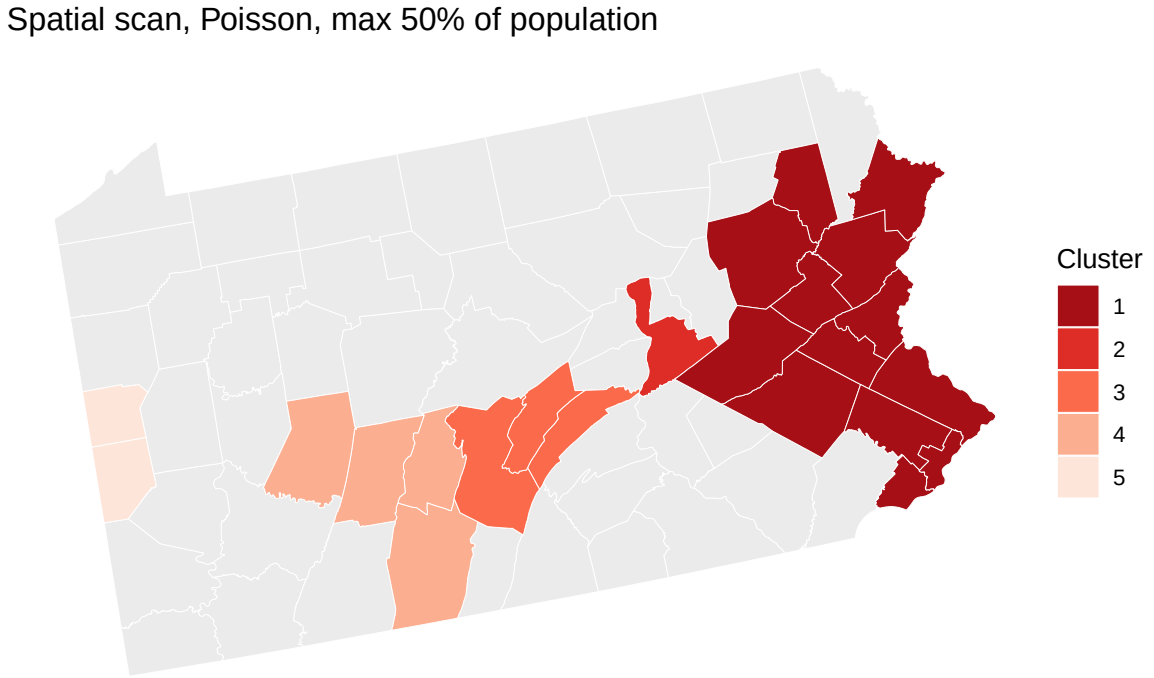
```
# table: one row per cluster
cluster_table <- function(kd) {
  clusters <- c(list(kd$most.likely.cluster), kd$secondary.clusters)
  C   <- sum(pa_covid$deaths_2020)                          # total deaths
  out <- NULL
  for (k in seq_along(clusters)) {
    x   <- clusters[[k]]
    row <- data.frame(
      cluster  = k,
      counties = paste(sort(pa_covid$NAME[x$location.IDs.included]), collapse = ", "),
      observed = x$number.of.cases,
      expected = round(x$expected.cases),
      SMR      = round(x$SMR, 2),                             # inside vs state rate
      RR       = round(x$SMR / ((C - x$number.of.cases) / (C - x$expected.cases)), 2),  # insi
      LLR      = round(x$log.likelihood.ratio, 1),
      p_value  = x$p.value)
    out <- rbind(out, row)                                   # add the row
  }
  out
}

# map: grey counties, clusters in red (dark = most likely)
cluster_map <- function(kd, title) {
  clusters <- c(list(kd$most.likely.cluster), kd$secondary.clusters)
  pa_covid$cluster <- NA                                     # no cluster yet
  for (k in seq_along(clusters)) {
    pa_covid$cluster[clusters[[k]]$location.IDs.included] <- k
  }
  ggplot() +
    geom_sf(data = pa_covid, fill = "grey92", colour = "white", linewidth = 0.15) +
    geom_sf(data = filter(pa_covid, !is.na(cluster)),
            aes(fill = factor(cluster)), colour = "white", linewidth = 0.15) +
    scale_fill_brewer(palette = "Reds", direction = -1, name = "Cluster") +
    labs(title = title) +
    theme_void()
}
```

```
tab50 <- cluster_table(kd50)
knitr::kable(tab50)                             # print as a table
```

cluster	counties	observed	expected	SMR	RR	LLR	p_value
1	Berks, Bucks, Carbon, Delaware, Lackawanna, Lehigh, Luzerne, Monroe, Montgomery, Northampton, Philadelphia, Pike, Schuylkill	8250	7093	1.16	1.34	167.9	0.001
2	Northumberland	226	114	1.99	2.00	43.3	0.001
3	Huntingdon, Juniata, Mifflin	253	145	1.74	1.75	33.0	0.001
4	Bedford, Blair, Cambria, Indiana	642	481	1.34	1.35	25.3	0.001
5	Beaver, Lawrence	378	312	1.21	1.22	6.6	0.045

```
cluster_map(kd50, "Spatial scan, Poisson, max 50% of population")
```



The R results should match the SaTScan results file closely. Small differences can arise because SaTScan measures distance on the globe from latitude and longitude, whereas here we use projected kilometres, and because the Monte Carlo p-values depend on the random seed.

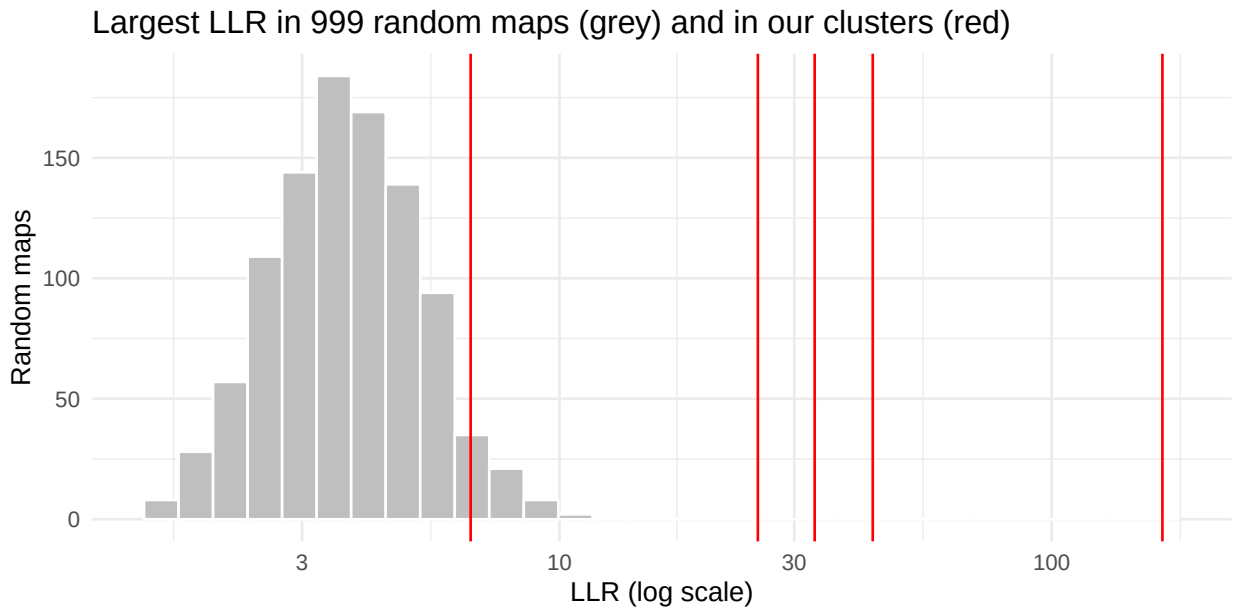
Where does the p-value come from?

`kulldorff()` keeps the largest LLR of each of the 999 random maps from Idea 3. Plotting them shows the Monte Carlo test at work.

```
sims <- data.frame(LLR = kd50$simulated.log.lkhd)   # one value per random map

ggplot(sims, aes(x = LLR)) +
  geom_histogram(bins = 30, fill = "grey75", colour = "white") +
```

```r
  geom_vline(xintercept = tab50$LLR, colour = "red") +      # our clusters
  scale_x_log10() +                                         # values run from 1 to 170
  labs(title = "Largest LLR in 999 random maps (grey) and in our clusters (red)",
       x = "LLR (log scale)", y = "Random maps") +
  theme_minimal()
```



The largest LLR produced by any random map was 12.4. Every cluster with $p = 0.001$ lies to the right of all the grey bars: no random map beats it, so $p = 1/1000$, the smallest value 999 replications can give. A cluster whose red line falls among the grey bars has a larger p-value.

Reading the result

The most likely cluster joins 13 counties in the east: Berks, Bucks, Carbon, Delaware, Lackawanna, Lehigh, Luzerne, Monroe, Montgomery, Northampton, Philadelphia, Pike, Schuylkill. Together they hold 44% of the state's population. They had 8,250 deaths against 7,093 expected: an SMR of only 1.16, or 16% above the state rate, and a relative risk of 1.34 against the rest of Pennsylvania. Yet its LLR, 167.9, is by far the largest. That is Idea 2 again: a modest excess spread over several million people amounts to about 1,157 extra deaths.

Look back at the SMR map: none of these counties is in the darkest classes, and Monroe and Pike are even below 1. They are in the cluster because the circle that captures the large excess around Philadelphia sweeps them in. The cluster's outline is approximate; its core is what matters.

The relative risk is larger than the SMR because it compares the inside of the circle with the *outside*, and the outside had fewer deaths than expected. SaTScan reports both.

There is no magic in the LLR. The formula from Idea 2, applied to the cluster's totals, gives the same value as `kulldorff()`, up to a small rounding difference inside the package:

```r
c_in <- kd50$most.likely.cluster$number.of.cases    # observed inside
E_in <- kd50$most.likely.cluster$expected.cases     # expected inside
c_in * log(c_in / E_in) + (C - c_in) * log((C - c_in) / (C - E_in))
```

```
[1] 168.0353
```

The secondary clusters are smaller and more intense: Northumberland on its own, a central group around Mifflin and Juniata, and a west-central group around Cambria and Blair, with SMRs between about 1.3 and 2. The last one, Beaver, Lawrence, has $p = 0.045$, close to the 0.05 line.

How sensitive is the answer to the window size?

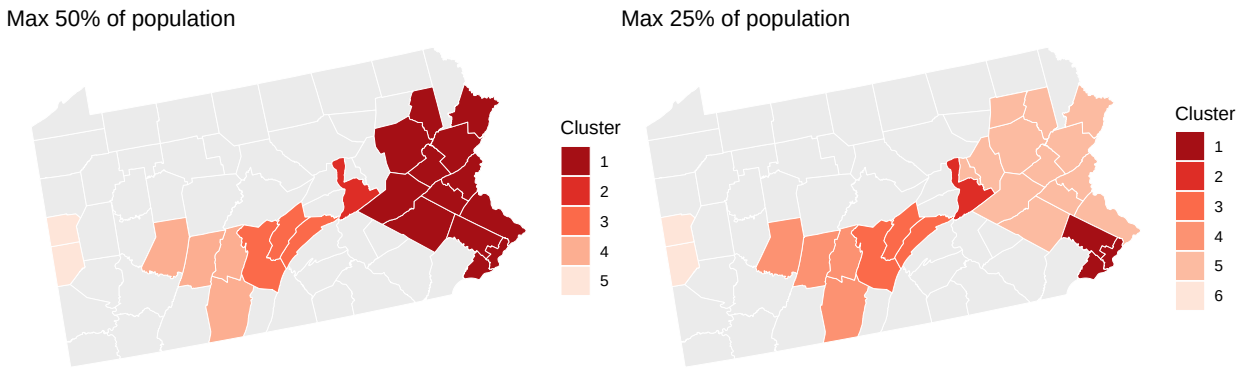
The 50% maximum is SaTScan's default, not a rule. The most likely cluster above holds 44% of Pennsylvania's population, which is hard to act on, so we repeat the scan with a 25% maximum.

```r
set.seed(374)
kd25 <- kulldorff(geo             = cen_km,
                  cases           = pa_covid$deaths_2020,
                  population      = pa_covid$Population,
                  expected.cases  = pa_covid$E,
                  pop.upper.bound = 0.25,                # the only change
                  n.simulations   = 999,
                  alpha.level     = 0.05,
                  plot            = FALSE)

tab25 <- cluster_table(kd25)
knitr::kable(tab25)
```

cluster	counties	observed	expected	SMR	RR	LLR	p_value
1	Delaware, Montgomery, Philadelphia	4548	3734	1.22	1.30	110.5	0.001
2	Northumberland	226	114	1.99	2.00	43.3	0.001
3	Huntingdon, Juniata, Mifflin	253	145	1.74	1.75	33.0	0.001
4	Bedford, Blair, Cambria, Indiana	642	481	1.34	1.35	25.3	0.001
5	Berks, Bucks, Carbon, Columbia, Lackawanna, Lehigh, Luzerne, Monroe, Montour, Northampton, Pike, Schuylkill, Wyoming	3827	3497	1.09	1.12	19.5	0.001
6	Beaver, Lawrence	378	312	1.21	1.22	6.6	0.041

```
# the two maps side by side
cluster_map(kd50, "Max 50% of population") +
  cluster_map(kd25, "Max 25% of population")
```



With the smaller window, the most likely cluster shrinks to Delaware, Montgomery, Philadelphia (SMR 1.22, LLR 110.5). The other eastern counties now appear further down the list as a separate, weaker cluster, while Northumberland and the central and west-central clusters are unchanged. Clusters that survive a change in window size deserve more confidence than those that do not.

Scan statistic or LISA?

	Local Moran's I (LISA)	Spatial scan (SaTScan)
Question	Is this county similar to its neighbours?	Is there a group of counties with more deaths than expected?
Works on	the SMR, a ratio: every county counts equally	counts, observed against expected: large populations weigh more
Neighbourhood	fixed (queen contiguity)	circles of every size, up to the cap
Tests	one per county, so a multiple-testing adjustment is needed	one test for the whole map; the adjustment is built in
Finds	High–High, Low–Low and outliers	high-rate clusters (low rates if asked)
Result	a map of significant counties	a ranked list of clusters with p-values

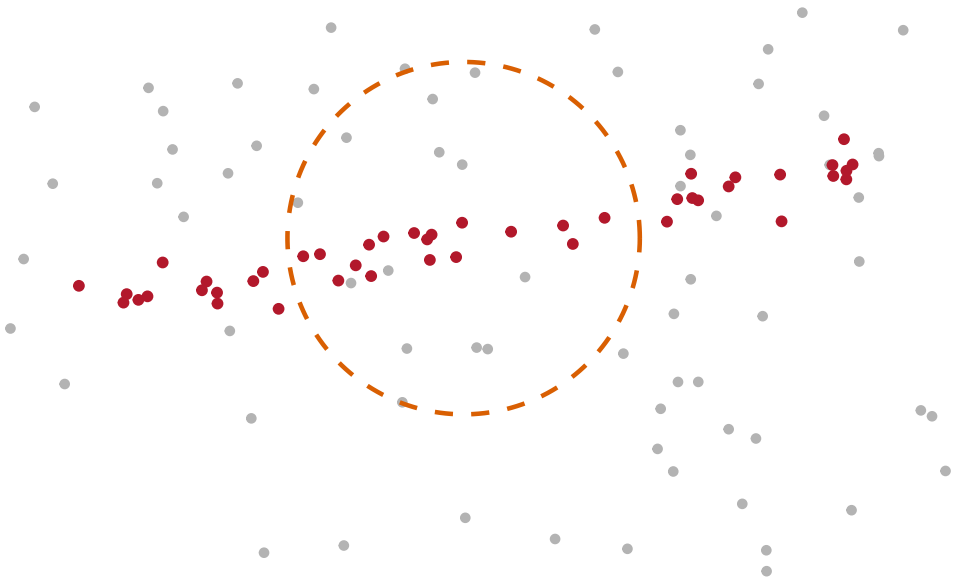
The two answer different questions, so they need not agree.

Limits to keep in mind

Circular windows fit compact clusters. A cluster stretched along a river valley or a highway is either cut short by the circle or diluted by the empty land around it. SaTScan's elliptic window helps.

```
set.seed(7)
valley <- tibble(x = runif(45, 1, 9)) |>
  mutate(y = 2.5 + 0.2 * x + rnorm(45, 0, 0.2))                # cases along a valley
backgr <- tibble(x = runif(70, 0, 10), y = runif(70, 0, 6))    # background cases
circle <- tibble(t = seq(0, 2 * pi, length.out = 200),
                 x = 5 + 1.8 * cos(t), y = 3.5 + 1.8 * sin(t))

ggplot() +
  geom_point(data = backgr, aes(x, y), colour = "grey70", size = 1.2) +
  geom_point(data = valley, aes(x, y), colour = "#B2182B", size = 1.4) +
  geom_path(data = circle, aes(x, y), colour = "#D95F02", linewidth = 0.8,
            linetype = "dashed") +
  coord_equal() +
  theme_void()
```



- **Window size** changes the answer, as we saw. Report the cap you used, and ideally more than one.

- **Edges**: the eastern cluster runs along the New Jersey state line and touches New York and Delaware. Deaths across the state line are not in our data, so a cluster that continues beyond Pennsylvania is cut at the border.
- **Centroids**: a county enters a circle when its centroid does, whatever its size or where its people live.
- **Expected counts** here use one statewide rate. Without age adjustment, a cluster may only reflect an older population. SaTScan can adjust for age groups if the case and population files include them; in R we would pass age-adjusted expected counts.
- **A significant cluster is a signal, not an explanation.** It says where to look; it does not say why. County-level clusters also say nothing about individuals.

Take-aways

Key points

1. The scan statistic moves circles of many sizes over the map, looking for a group of areas with more cases than expected.
2. Each circle gets an LLR comparing inside with outside; the largest LLR is the most likely cluster.
3. Monte Carlo replications give one p-value for the whole search, so no further multiple-testing adjustment is needed.
4. The scan works with counts: a small excess in a large population can outweigh a high SMR in a small county.
5. SaTScan needs three text files linked by a location ID; `kulldorff()` gives the same answer in R.
6. Report the window size, and treat a cluster as a place to look, not a cause.

Questions for discussion

1. The most likely cluster at 50% has an SMR of about 1.16. Would you report it to the Department of Health as a COVID-19 "hotspot"? What would you say about it?
2. Mifflin has the highest SMR in the state, yet on its own its LLR is lower than Philadelphia's. Why? For public-health action, which matters more: a high ratio or a large number of excess deaths?
3. The weakest secondary cluster has a p-value close to 0.05. Find its LLR on the Monte Carlo histogram. How much weight would you give it?
4. Which window size would you choose for a public-health report, and why?
5. How would the results change if expected deaths were adjusted for age?

Resources

- SaTScan background page for this course.
- SaTScan software and user guide: https://www.satscan.org.
- Kulldorff, M. (1997). A spatial scan statistic. *Communications in Statistics — Theory and Methods*, 26(6), 1481–1496.
- Rogerson & Yamada, *Statistical Detection and Surveillance of Geographic Clusters*, Chapter 5.